Inspiring Excellence

Department of Computer Science and Engineering

Course: CSE447: Cryptography and Cryptanalysis

Semester: Spring 2026

Project Report

Title: AutoVibe Crypto

Submitted To: Sajid Imam Mahir and Abdullah Khondoker

Group No: 06

Section: 01

Submission Date: [08 May 2026]

Group Members

No.	Full Name	Student ID
1	Humaira Ayesha	23301683

Table of Contents

1. Introduction and System Overview

This report documents the design, implementation, and security analysis of the CSE447 Lab Project. The system is a secure web application integrating multiple cryptographic protocols as required by the course specification. All encryption algorithms have been implemented from scratch without relying on built-in framework encryption functions.

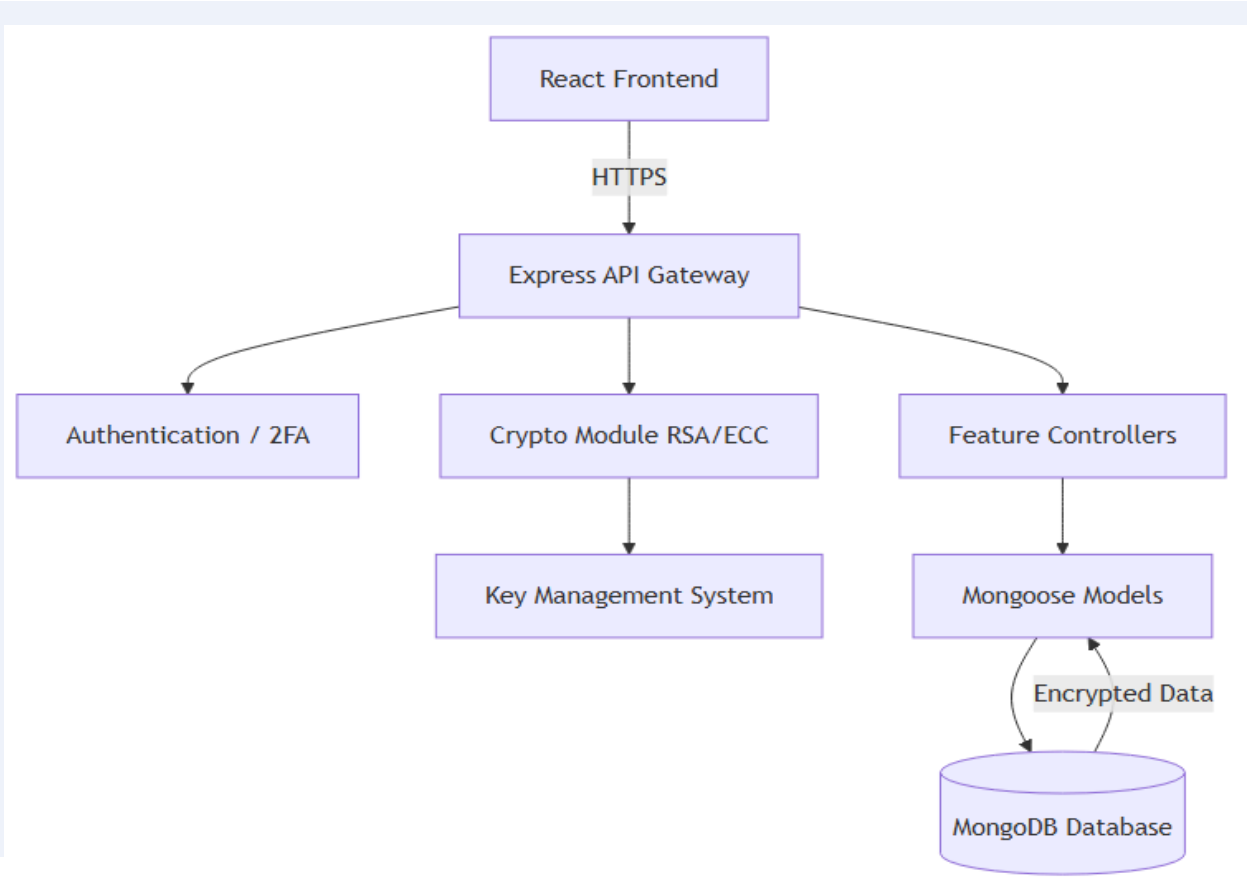
1.1 Project Overview

AutoVibe Crypto is a full-stack e-commerce platform designed for selling and managing car accessories while demonstrating practical cryptographic security. The platform includes common e-commerce features such as product browsing, product details, cart management, order placement, reviews, user profile management, and user posts. The main contribution of the project is that the application does not store important user and content data in plaintext. Instead, it applies a custom cryptography layer where RSA protects sensitive identity and order information, ECC protects application content such as products, reviews, and posts, SHA-256 supports password hashing and deterministic lookup, and HMAC-SHA256 verifies data integrity. This makes the project both a functional web application and a practical implementation of core CSE447 cryptography concepts.

1.2 Technology Stack

- Frontend: React with Vite is used to build the client-side interface, while Redux Toolkit manages application state such as user login status, cart data, products, orders, and profile information. TailwindCSS, Bootstrap, and Material-UI are used together to create responsive pages, forms, dashboards, cards, modals, and reusable UI components.
- Backend: Node.js and Express.js implement the REST API layer. The backend handles authentication, registration, product management, order processing, post management, profile updates, 2FA verification, session-token generation, and all custom cryptographic operations before data is saved or returned.
- Database: MongoDB with Mongoose ODM stores users, products, orders, reviews, posts, OTP records, and cryptographic key metadata. Sensitive fields are saved as ciphertext, while helper fields such as emailHash and keyVersion are stored to support lookup and correct decryption without exposing plaintext.
- Other Tools: Multer is used for uploading product and profile-related images, while environment variables store application secrets such as database connection strings and Key Encryption Key configuration. These supporting tools are integrated without replacing the custom cryptographic implementations.

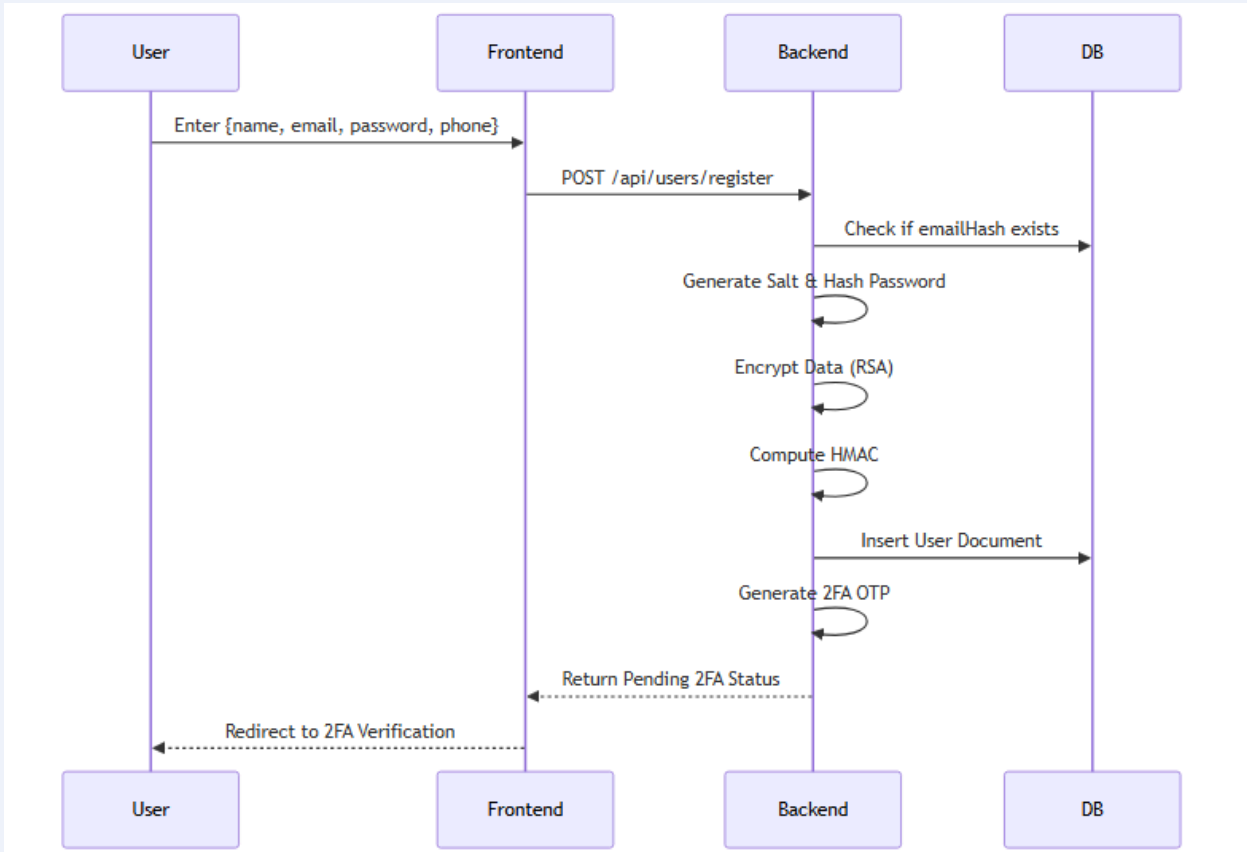
1.3 System Architecture Diagram



2. Login and Registration Module

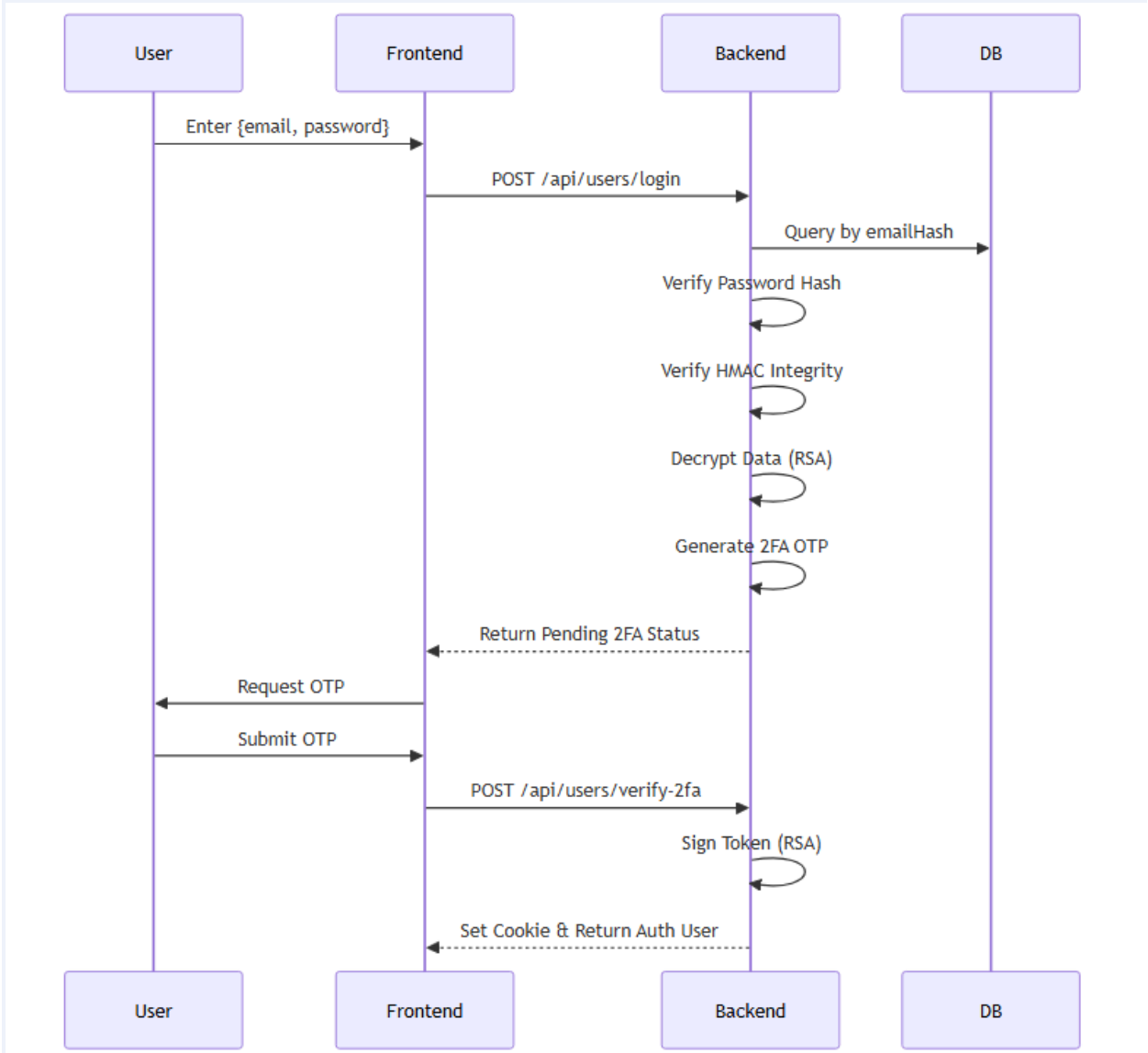
The system provides secure registration and login flows. New users supply credentials which are validated, encrypted, and persisted. During login, stored encrypted data is retrieved and decrypted for verification.

2.1 Registration Flow



The registration process starts when a new user submits required account details such as name, email, password, phone number, and optional address/profile information. The backend validates the input, checks required fields, verifies email format, and computes a deterministic SHA-256 emailHash to detect duplicate accounts without storing or searching by plaintext email. The password is then protected using a unique 32-byte salt and 10,000 SHA-256 iterations. After that, sensitive user fields such as name, email, phone, and address are encrypted using the active RSA USER_DATA public key. Finally, the system generates a data HMAC over the encrypted payload and stores the encrypted values, password hash string, emailHash, role, key version, and integrity tag in MongoDB.

2.2 Login Flow



The login process begins when the user submits email and password. The backend recomputes the SHA-256 emailHash from the email and uses that value to find the user record, so the application does not need to query by plaintext email. It then extracts the stored salt and iteration count, hashes the submitted password with the same 10,000-iteration SHA-256 process, and compares the result with the saved password hash. If the password is valid, the system does not immediately create a final session; instead, it triggers the OTP step and returns a pending2FA response. After the OTP is verified, the backend creates a custom RSA-signed session token and sends it as a secure cookie

2.3 Implementation Details

Requirement	Implementation Details
Login Module	*Users are authenticated by combining deterministic email lookup, salted password verification, two-factor OTP verification, and custom RSA-signed session*

	tokens. The login API computes emailHash from the submitted email, retrieves the encrypted user record, verifies the password using the stored salt and 10,000 SHA-256 iterations, triggers the OTP step, and creates a session only after both factors are valid.
Registration Module	*Registration collects user identity and account fields such as name, email, phone, password, and profile/order-related information. The backend validates required fields, checks for an existing emailHash, hashes the password with a unique salt, encrypts sensitive user attributes with RSA, generates an integrity HMAC, and saves only the protected values in MongoDB.*
Data Encrypted Before Storage	*Before storage, name, email, phone, profile address, and order shipping information are encrypted with RSA. Product title/name, product description, brand information, review content, and post title/content are encrypted with ECC. Passwords are not encrypted; they are salted and hashed with iterative SHA-256. Private cryptographic key material is protected separately using a Key Encryption Key.*
Data Decrypted on Retrieval	*On retrieval, the backend first verifies the stored HMAC to detect tampering. It then selects the correct key using key purpose and keyVersion/keyId, decrypts the required fields in memory, and returns plaintext only to authorized application flows such as login, profile view, product display, order details, or post listing. Plaintext decrypted values are not written back to the database.*

3. User Data Encryption and Decryption

All sensitive user information (e.g., username, email, contact info) is encrypted before storage using asymmetric encryption algorithms implemented from scratch, and decrypted upon retrieval.

3.1 Fields Encrypted

- Name: Encrypted using RSA
- Email: Encrypted using RSA (A deterministic SHA-256 emailHash is also maintained for fast lookups)
- Phone: Encrypted using RSA
- Shipping Address (Orders): Encrypted using RSA
- Product Details (Name/Desc/Brand): Encrypted using ECC

In this project, encryption is applied at the field level. For the User collection, name, email, phone, and profile address are encrypted using RSA under the USER_DATA key purpose; emailHash is kept

separately as a SHA-256 digest only for login lookup. For the Order collection, shipping address and customer contact details are encrypted using RSA under ORDER_DATA. For Product, Review, and Post collections, content-oriented fields such as product name, description, brand, review text, post title, and post content are encrypted using ECC under PRODUCT_DATA, REVIEW_DATA, or POST_DATA. Operational fields such as IDs, timestamps, stock counts, price, role, and status may remain plaintext where needed for filtering, authorization, and business logic.

3.2 Encryption Algorithm - RSA Implementation

RSA (Rivest-Shamir-Adleman) is implemented from scratch in the backend cryptography layer using native JavaScript BigInt arithmetic rather than framework-level encryption functions. The implementation is used for identity-heavy and highly sensitive fields because RSA naturally supports public-key encryption and signature-style verification in this project.

● Key Generation: Generates large primes p and q to compute modulus n = p * q. The public exponent e is typically 65537, and d is computed using the modular inverse.

● Encryption/Decryption: Handles big-integer modular exponentiation (M^e mod n and C^d mod n).

● Format: Data is converted to stringified arrays of large number blocks to avoid integer overflow issues in JavaScript.

The RSA workflow generates two large prime numbers p and q, computes n = p × q, calculates φ(n), selects the public exponent e, and derives the private exponent d using the modular inverse. During encryption, plaintext strings are converted into numeric blocks so each block remains smaller than n; each block is encrypted as C = M^e mod n using fast modular exponentiation. During decryption, the private exponent reconstructs M = C^d mod n and the blocks are converted back into text. The block-based format is stored as serialized ciphertext arrays, which avoids JavaScript number overflow and allows long strings such as emails, addresses, and names to be encrypted safely. In addition to field encryption, the RSA module is also used for signing custom session tokens under the SESSION_SIGNING key purpose.

3.3 Encryption Algorithm - ECC Implementation

Elliptic Curve Cryptography is implemented over the secp256k1 curve, defined by $y^2 = x^3 + 7 \bmod p$. It is used for application content because ECC provides strong security with comparatively compact key material and demonstrates a second asymmetric algorithm separate from RSA.

● Encryption Scheme: EC-ElGamal encryption.

● Operations: Point addition and scalar multiplication are written from scratch without any external BigInt libraries (using native JS BigInt).

● Ciphertext: Returns a tuple (R, S) representing points on the elliptic curve.

The ECC module implements the required curve arithmetic manually, including modular inverse, point addition, point doubling, scalar multiplication, public-key derivation, and ciphertext construction. The project follows an EC-ElGamal-style approach: for each encryption, a random scalar k is generated, an ephemeral point R = kG is produced, and a shared point is derived from the receiver public key. The plaintext is converted into numeric content blocks and protected using the derived curve point, producing ciphertext components that include curve points such as R and S. On decryption, the private scalar reconstructs the shared secret and recovers the original content. This

implementation uses native BigInt operations and avoids external cryptography libraries for the core curve calculations.

3.4 How Both Algorithms Are Used Differently

● RSA: Exclusively used for highly sensitive user-identifying data (USER_DATA), order shipping details (ORDER_DATA), and session token signing (SESSION_SIGNING).

● ECC: Applied to application content data such as product descriptions (PRODUCT_DATA), reviews (REVIEW_DATA), and user posts (POST_DATA).

RSA and ECC are intentionally separated by data purpose. RSA is used for smaller but highly sensitive personally identifiable and transactional fields, such as user identity, email, phone, address, shipping details, and session-token signing. ECC is used for larger application content, such as product details, reviews, and user posts. This separation satisfies the project requirement that the same algorithm is not used everywhere, and it also makes the security design easier to audit because each encrypted field is associated with a specific key purpose and algorithm.

4. Password Hashing and Salting

Passwords are never stored in plaintext. A cryptographic hash function combined with a random salt is applied before storage to prevent dictionary and rainbow-table attacks.

4.1 Hashing Algorithm Used

An iterative SHA-256 implementation acts as a Key Derivation Function (similar to PBKDF2), requiring 10,000 iterations to prevent brute-force cracking.

The project uses a custom SHA-256 implementation as the base hash function and applies it iteratively to act like a simple password-based key derivation process. SHA-256 was selected because it is a widely understood cryptographic hash function, produces a fixed 256-bit digest, and is appropriate for demonstrating one-way password protection in a cryptography course project. The repeated 10,000-iteration design increases the cost of brute-force attempts compared with a single hash, while still remaining practical for login verification in the web application.

4.2 Salt Generation

A 32-byte cryptographically secure random salt is generated for every user during registration. It is appended to the hash format and stored alongside it in the database (e.g., 10000salt$hash).

For every new user, the backend generates a unique 32-byte random salt before hashing the password. The salt is not secret; its purpose is to ensure that two users with the same password still produce different stored hashes and to make rainbow-table attacks ineffective. The final stored password value includes the iteration count, the salt, and the derived hash in one structured string, for example 10000salt$hash. This allows the login function to repeat the exact same hashing process later without needing to store the original password.

4.3 Verification Process

During authentication, the salt and iteration count are extracted from the stored string. The input password is mixed with the salt and iterated 10,000 times through SHA-256. If the output matches the stored hash, access is granted.

When a user attempts to log in, the stored password string is split into its iteration count, salt, and hash components. The submitted password is combined with the extracted salt and passed through the same iterative SHA-256 routine for the same number of rounds. The newly generated hash is then compared with the stored hash. If the values match, the password step is valid; if not, the system rejects the login without revealing whether the email or password was the incorrect part. This verification method ensures that passwords remain non-reversible even to someone with database access.

5. Two-Factor Authentication (2FA)

The system enforces two-step verification: the user must pass both primary credential validation and a second authentication factor before a session is granted.

5.1 2FA Method

A time-limited One-Time Password (OTP) method is used as the second authentication factor. After the password step succeeds, the backend does not immediately issue a session. Instead, it creates or validates an OTP challenge, stores only the hashed OTP value, assigns an expiration time, and responds with a pending2FA flag. The frontend then redirects the user to a verification screen where the user enters the 6-digit code. This ensures that knowing the account password alone is not enough to access the account.

In the implemented flow, the second factor is a 6-digit OTP associated with the user account. The plaintext OTP is never stored directly; the server stores a SHA-256 hash of the OTP together with an expiry timestamp in the Otp collection. During verification, the submitted code is hashed with the same SHA-256 function and compared with the stored otpHash. If the record is missing, expired, or mismatched, authentication fails. If the code is correct, the OTP record is deleted so it cannot be reused, and only then does the backend issue the RSA-signed session cookie.

5.2 Code Snippet

```
// Verification Snippet
export const verifyTwoFactor = async (req, res) => {
  const { userId, otpCode } = req.body;
  const otpRecord = await Otp.findOne({ user: userId });

  if (!otpRecord || otpRecord.expiresAt < Date.now()) {
    return res.status(400).json({ message: "OTP expired or invalid" });
  }
```

```
  const hashedInputOtp = sha256(otpCode);
  if (hashedInputOtp === otpRecord.otpHash) {
    // OTP verified successfully, proceed with issuing RSA-signed token
    await Otp.deleteOne({ _id: otpRecord._id });
    generateToken(res, userId); // Assigns Session Cookie
    res.json({ success: true });
  } else {
    res.status(400).json({ message: "Incorrect OTP code" });
  }
};
```

[Paste relevant 2FA implementation code here.]

6. Key Management Module

A dedicated Key Management Module handles the full lifecycle of cryptographic keys: secure storage, and rotation.

6.1 Key Storage Security

Private keys (d component in RSA, d scalar in ECC) are encrypted at rest using a master Key Encryption Key (KEK) before being stored in MongoDB. The keys are only decrypted in memory during the application lifecycle.

The Key Management Module stores key metadata separately from normal application records. Each key entry contains information such as algorithm type, key purpose, version, status, public component, and encrypted private component. Public keys can be used for encryption and verification, but private key material is protected before storage using a master Key Encryption Key configured through the server environment. The backend decrypts private keys only temporarily in memory when decryption or signing is required. Private keys are never exposed to the frontend, never printed in API responses, and are accessible only through protected backend logic.

6.2 Key Rotation Policy

Keys are versioned. New keys can be dynamically generated and labeled as ACTIVE. Older keys are transitioned to a ROTATED state, meaning they can no longer be used for new encryptions, but are kept available for decrypting historical records (preventing data loss).

The project uses versioned key rotation. At any time, one key for a specific purpose, such as USER_DATA or POST_DATA, is marked ACTIVE and is used for new encryption. When a new key is generated, the previous active key is changed to ROTATED. A rotated key is blocked from encrypting new records, but it is preserved for decrypting older records that were encrypted with that version.

Encrypted documents store their keyVersion or keyId, so the backend can select the correct historical key. This prevents data loss while still allowing newer data to move to fresh key material.

7. Post and Profile Management

Users can create, view, and edit posts, as well as view and update their profiles. All post and profile data is automatically encrypted before storage and decrypted on retrieval.

7.1 Post Module

When creating or updating posts, the title and content are encrypted using ECC algorithms. A computed HMAC ensures the post's integrity. When fetching public or private posts, the server decrypts the content on-the-fly.

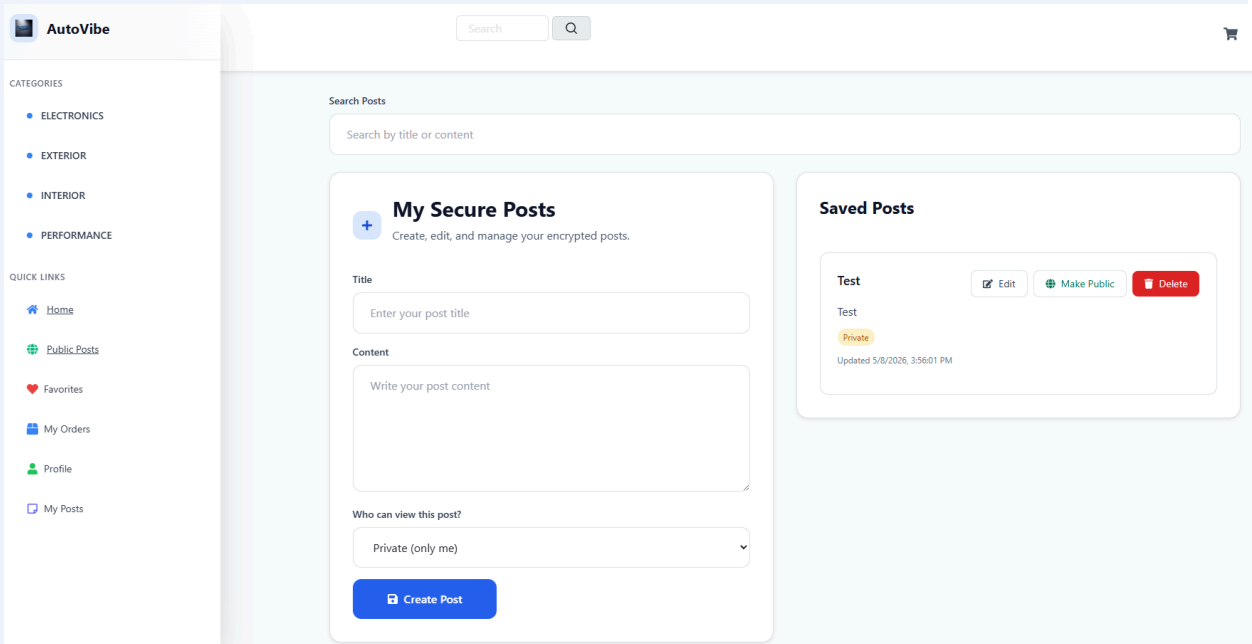
For post creation, the user submits the post title and content from the frontend. The backend verifies the session, checks the user role, encrypts the title and content using the active ECC POST_DATA key, computes an HMAC over the encrypted payload, and stores the ciphertext in MongoDB. During post listing or detail view, the backend verifies the HMAC first, decrypts the ECC ciphertext in memory, and returns readable post data to the frontend. During update, the same process is repeated: the new title/content is encrypted again and a new HMAC is generated. Ownership and role checks ensure that a regular user can manage only their permitted posts, while an admin can perform broader moderation tasks.

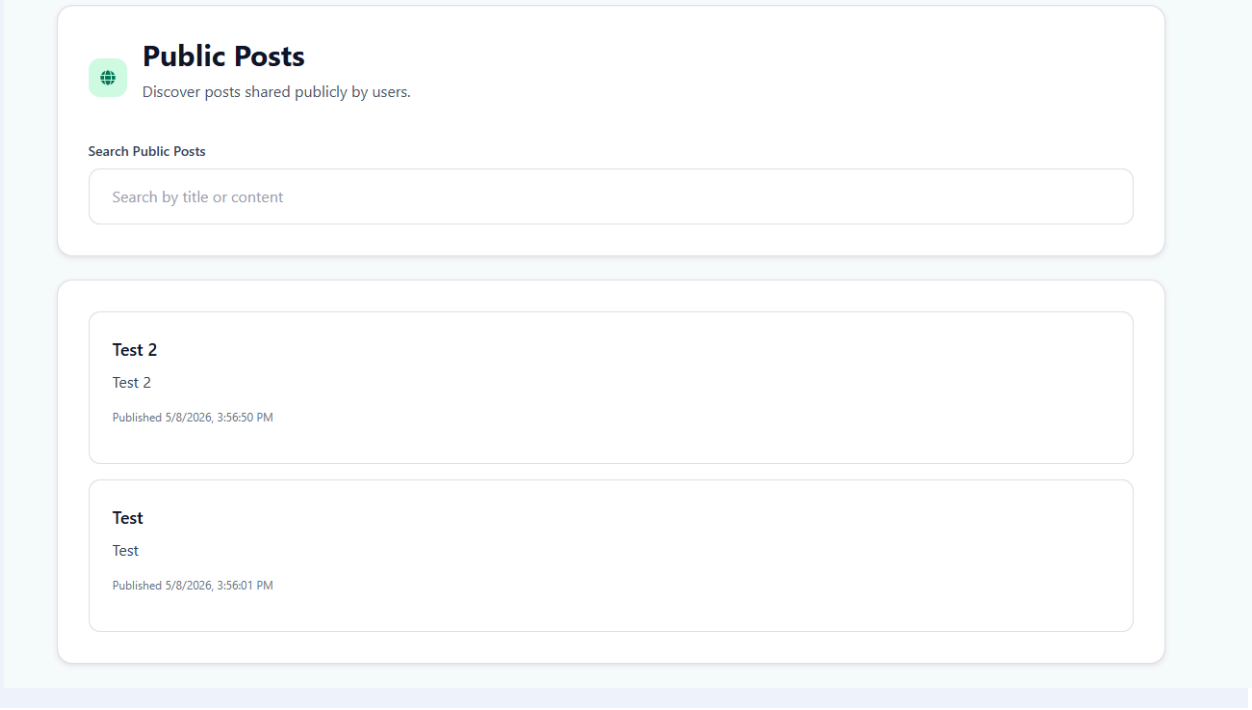
7.2 Profile Module

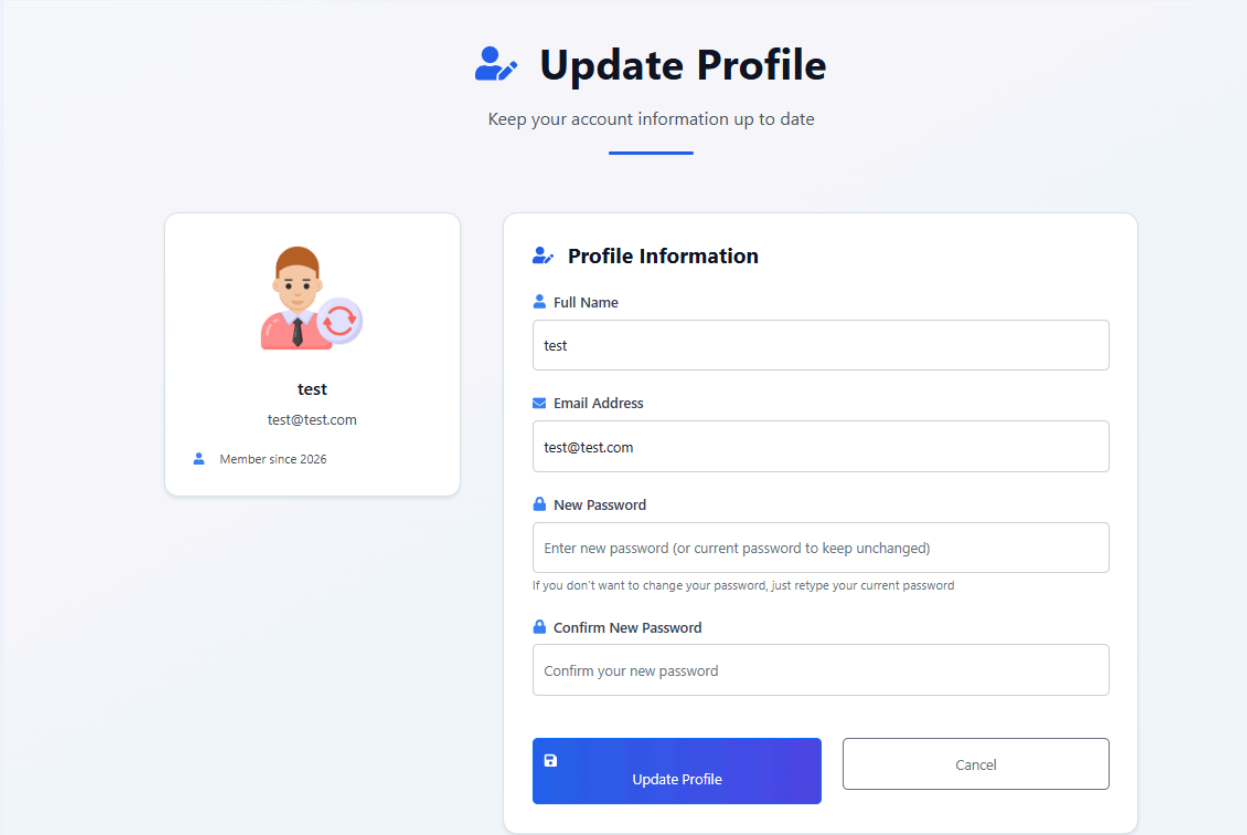
Profile fields (Name, Phone, Address) are protected via RSA encryption. Updates to the profile generate a brand-new HMAC to lock the integrity of the newly modified data.

For profile viewing, the authenticated user request is checked through the session middleware. The backend retrieves the user record, verifies the stored data HMAC, selects the correct RSA key version, decrypts protected fields such as name, email, phone, and address in memory, and sends the readable profile data to the frontend. For profile update, the submitted values are validated and then re-encrypted with the active RSA USER_DATA key. A fresh HMAC is generated after encryption, ensuring that any later unauthorized database modification can be detected.

7.3 Screenshots


AutoVibe
Search
CATEGORIES
ELECTRONICS
EXTERIOR
INTERIOR
PERFORMANCE
QUICK LINKS
Home
Public Posts
Favorites
My Orders
Profile
My Posts
Search Posts
Search by title or content
My Secure Posts
Create, edit, and manage your encrypted posts.
Title
Enter your post title
Content
Write your post content
Who can view this post?
Private (only me)
Create Post
Saved Posts
Test
Edit
Make Public
Delete
Test
Private
Updated 5/8/2026, 3:56:01 PM


Public Posts
Discover posts shared publicly by users.
Search Public Posts
Search by title or content
Test 2
Test 2
Published 5/8/2026, 3:56:50 PM
Test
Test
Published 5/8/2026, 3:56:01 PM



8. Data Storage Security

All critical data - user information, posts, and cryptographic keys - is stored in encrypted form to prevent plaintext access even in the event of a database compromise.

8.1 Evidence of Encrypted Storage


```
_id: ObjectId('69e269629c7efbb95dca8c8e')
name : "["7367a0bc98f5a5dc16b24f2793142922592e36e8fcd896ef1fe4dd1228e7e72e4593…"
email : "["6c9722e97f075d13fbb16b9b9b1857f6c0604dfbdc169338c16db2ff5e5c676b99f5…"
emailHash : "f660ab912ec121d1b1e928a0bb4bc61b15f5ad44d5efdc4e1c92a25e99b8e44a"
phone : "["625714075564a20eec0d54ff847538f39054f1534afee1be85dd30fe3e0a3d736297…"
password : "$10000$1be7de11b4851ad36e3d7ef4530476aa$35c49dc1fb05a426bb345b5f44401a…"
role : "user"
isAdmin : false
isArtist : false
▸ favoriteProducts : Array (empty)
points : 0
▸ membership : Object
twoFactorEnabled : true
dataHmac : "1216e4fe30cc2a31fc8f3802045746e85b856dba8770f80bd0711703e2a8a7f1"
encryptionKeyVersion : 1
createdAt : 2026-04-17T17:09:54.890+00:00
updatedAt : 2026-04-17T18:01:04.015+00:00
__v : 0
```

The encrypted-storage evidence demonstrates that sensitive application data is not saved as readable plaintext in MongoDB. User identity fields, order shipping fields, product/review/post content, and private cryptographic key components appear as ciphertext, serialized encrypted blocks, hashes, HMAC values, key versions, and metadata. This means that even if an attacker directly views the database collections, they cannot immediately read names, emails, phone numbers, addresses, post content, or protected product descriptions without the correct private keys and Key Encryption Key.

9. Message Authentication Code (MAC)

Message Authentication Codes (MACs) are used to verify the integrity of stored data and detect any unauthorized modifications. The system implements CBC-MAC or HMAC as required.

9.1 MAC Algorithm Used

HMAC-SHA256 is used. The implementation processes 64-byte blocks with inner (ipad) and outer (opad) padding combined with a secret server key.

The system uses HMAC-SHA256 rather than CBC-MAC because the project already implements SHA-256 from scratch and because HMAC is appropriate for variable-length structured data such as encrypted user records, products, reviews, orders, and posts. The HMAC implementation first normalizes the secret key to the SHA-256 block size, applies the inner padding value ipad, hashes the padded key with the message, then applies the outer padding value opad and hashes again. The final output is stored as dataHmac beside the encrypted payload. Since the HMAC is computed over ciphertext and important metadata, unauthorized changes to either encrypted data or related fields can be detected.

9.2 Integrity Verification Flow

When a user profile, product, or order is retrieved from the database, the backend recomputes the HMAC using the encrypted payload. A constant-time comparison is executed against the dataHmac stored in the database. If they differ, the system throws a data tampering alert.

MAC verification is performed whenever protected records are read or used in a critical operation. For example, before returning a profile, order, product, review, or post, the backend recomputes the HMAC from the stored encrypted payload and compares it with the saved dataHmac. A constant-time comparison is used to reduce timing-based leakage. If the values do not match, the system treats the record as tampered and stops normal processing instead of decrypting and returning corrupted data. When a protected record is created or updated, a fresh HMAC is generated immediately after encryption and saved with the record.

10. Role-Based Access Control (RBAC)

Role-Based Access Control defines distinct privilege levels for Administrators and Regular Users, ensuring that sensitive operations are restricted appropriately.

10.1 Roles Defined

● Admin: Has full access to user management, product additions, orders filtering, sales dashboards, and key management functionalities.

● User: Can browse products, manage their cart, create orders, leave reviews, and publish posts.

[List all roles in the system (e.g., Admin, User) and their high-level responsibilities.]

10.2 Permission Matrix

Operation / Resource	Admin	Regular User
Browse Products	✔	✔
Manage Profile	✔	✔
Create Order	✔	✔
Create Posts	✔	✔
Manage Cryptographic Keys	✔	✘
View Sales Reports	✔	✘
Browse Products	✔	✘
Manage Profile	✔	✘

(Add or remove rows as appropriate for your system. Tick marks above are placeholders.)

11. Secure Session Management

Authentication tokens and session identifiers are managed securely to prevent session hijacking, fixation, and replay attacks.

11.1 Token Signing / Verification

The application completely abandons standard JWT libraries. Instead, tokens are formed as Base64 JSON structures and signed via the custom RSA Implementation.

During incoming requests, the AuthMiddleware verifies the RSA signature using the SESSION_SIGNING public key. The middleware also hashes the user's IP address and User-Agent with SHA-256 to create a request fingerprint. If a token is copied to another device or browser, the fingerprint check helps detect the mismatch and prevents the copied token from being accepted silently.

After successful password and OTP verification, the backend creates a custom session token instead of using a standard JWT library. The token payload contains the user ID, role, issue time, expiry time, and a SHA-256 fingerprint derived from request-specific details such as IP address and User-Agent. This JSON payload is Base64 encoded and signed using the custom RSA SESSION_SIGNING private key. For every protected request, the authentication middleware decodes the token, verifies the RSA signature with the matching public key, checks expiration, recomputes the fingerprint, and loads the user context only if all checks pass. This design protects against token tampering and reduces the risk of successful session hijacking.

12. GitHub Repository and Project Structure

Field	Details
GitHub Repository URL	*https://github.com/humu14/AutoVibe_Crypto*

12.1 Repository Structure

```
AutoVibe_Crypto/
├── backend/
│   ├── config/        Database connection and environment setup
│   ├── controllers/   API logic for auth, users, products, orders, posts, reviews, 2FA, and keys
│   ├── crypto/        Custom RSA, ECC, SHA-256, HMAC, password hashing, and key utilities
│   ├── middleware/    Authentication, role authorization, error handling, and session verification
│   ├── models/        Mongoose schemas for users, products, orders, posts, OTPs, reviews, and keys
│   ├── routes/        Express route definitions mapped to backend controllers
│   └── utils/         Token generation, helper functions, and shared backend utilities
├── frontend/
│   └── src/
│       ├── components/  Reusable UI components
│       ├── screens/     Pages for home, products, cart, login, register, profile, posts, admin, and
orders
│       ├── slices/      Redux Toolkit API slices and state logic
│       └── utils/       Frontend helper functions
├── uploads/           Uploaded images and static assets
├── .env               Local environment variables
├── package.json       Dependency and script configuration
└── README.md          Setup guide and cryptography usage documentation
```

12.2 README Overview

The README.md file acts as a comprehensive technical manifest. It outlines the technology stack, environment variable requirements (such as defining KEY_ENCRYPTION_* keys and MongoDB connection string), setup scripts, and a detailed mapping of exactly where cryptography is utilized across backend routes and frontend components.

[Summarise what your README.md contains: setup instructions, environment requirements, how to run the project locally.]

13. Conclusion

Building AutoVibe Crypto as a full-stack e-commerce platform with custom cryptographic functions presented several important challenges. The most significant difficulties were implementing large-number arithmetic in JavaScript, keeping RSA and ECC ciphertext formats consistent, protecting private keys, supporting key rotation without losing access to older records, and integrating security checks into normal e-commerce workflows such as login, checkout, product display, reviews, posts, and profile management. Through iterative development and debugging, the final system demonstrates how cryptographic algorithms can be connected to real application requirements. The project replaces common ready-made security libraries with custom RSA, ECC, SHA-256, HMAC, salted password hashing, OTP verification, RBAC, and RSA-signed session handling. As a result, AutoVibe Crypto satisfies the major CSE447 requirements while showing the practical importance of confidentiality, integrity, authentication, authorization, and secure key management in a modern web application.